Karpathy Modelleri FPGA’de · konuşma metni

Nasıl kullanılır: Her başlık bir slayta karşılık gelir; numaralar slayt numarasıdır. Aynı metin sunumda her slaytın konuşmacı notunda da var.

Akış: 1: Konu, sonuçların özeti ve yol haritası. · 4: Hedef: bilgisayarsız çalışan bir FPGA tasarımı ve kısıtları. · 7: Karpathy model ailesi: 9 mimari, stories260K ve TinyStories sürümleri. · 11: Integer-only aritmetik: quantization, requant, softmax, norm ve tablolar. · 18: PyTorch modelinden mikrokoda ve bellek imajlarına. · 21: Bütün modelleri çalıştıran mikrokodlu ortak engine: zoo_core. · 26: Kart sistemi: üretim döngüsü, integer sampling, VGA ekranı ve kart üstü kontrol. · 31: Düşük bitli weight’ler: Larq, LittleBit ve 1-bit matris-vektör birimi. · 37: Doğrulama zinciri ve bulunan hatalar. · 41: Açık kaynak yerleşim ve yönlendirme: Fmax, kaynaklar ve hız. · 46: Karşılaştırma, sınırlar ve sonraki adımlar.

---

1. Karpathy modelleri FPGA’de

Hocam merhaba. İlk bölümde minGPT’nin içini adım adım anlatmış ve FPGA açısından potansiyelini tartışmıştım. Bu bölüm implementasyonun kendisi. Karpathy’nin dil modeli ailesinden dokuz farklı mimariyi, ayrıca Karpathy’nin kendi eğittiği stories260K modelini, tek bir FPGA tasarımında, tamamen integer aritmetikle ve hiçbir bilgisayara bağlı olmadan çalıştırıyoruz. Sırasıyla şunları anlatacağım: hedefimiz ve kısıtlarımız, model ailesi, floating point kullanmadan bir language model’i nasıl hesapladığımız, modeli donanımın anlayacağı mikrokoda çeviren derleyici, bütün modelleri çalıştıran ortak engine, kartın üzerindeki üretim döngüsü ve ekran, hocamın önerdiği LittleBit ve Larq ile düşük bitli weight’ler, bunların hepsini nasıl doğruladığımız ve son olarak devrenin çipe yerleştirilip hangi hızda çalışabildiği. Teknik terimleri yine İngilizce bırakıyorum.

2. Sonuç: 18 model tek bir integer engine’de, bilgisayarsız ve Python referansıyla bit bit aynı çalışıyor

Önce sonucu söyleyeyim, sonra oraya nasıl geldiğimizi anlatayım. Karpathy’nin dil modeli ailesinden dokuz mimariyi aldık: en basiti bigram, en moderni nanochat. Bunlara Karpathy’nin kendi eğittiği stories260K modelini, aynı mimarilerin TinyStories’te eğittiğimiz beş sürümünü ve hocamın önerisiyle denediğimiz üç düşük bitli modeli ekleyince 18 model ediyor. Bu 18 modelin hepsi tek bir donanım tasarımında çalışıyor: modelden modele değişen şey devre değil, BRAM’e yüklenen mikrokod ve weight’ler. Hesabın tamamı integer; softmax, normalization ve hatta bir sonraki token’ı seçen sampling de dahil hiçbir yerde floating point yok. En önemlisi doğruluk: kartın bütününü cycle cycle simüle ettiğimizde, 18 modelin hepsinde üretilen her token, ekrana yazılan her karakter ve VGA çıkışındaki her piksel, Python’da yazdığımız integer referansla bit bit aynı. Somut bir örnek: Karpathy’nin 260 bin parametreli modeli kartın ekranına 256 token’lık bir hikâye yazıyor ve bu hikâye Python’daki modelin yazdığıyla token token aynı.

3. Sunum, bir modelin PyTorch’tan çipe giden yolunu dokuz adımda izliyor

Sunumun akışı, bir modelin PyTorch’tan çipe giden yolunu izliyor. Önce hedefimizi ve kartın bize koyduğu kısıtları anlatacağım. Sonra model ailesini tanıtacağım: dokuz mimari ne ortak, ne farklı.

Üçüncü bölüm işin matematiği: floating point olmadan bir language model'i nasıl hesaplıyoruz. Dördüncü bölüm derleyici: bir PyTorch modelini donanımın okuyacağı mikrokoda ve bellek imajlarına nasıl çeviriyoruz. Beşinci bölüm bütün modelleri çalıştıran ortak engine, altıncı bölüm de engine'in etrafındaki kart sistemi: üretim döngüsü, sampling ve ekran. Turuncu olan son üç bölüm projenin bu aşamada eklenen kısımları: hocamın önerdiği LittleBit ve Larq ile düşük bitli weight'ler, her şeyi nasıl doğruladığımız ve devrenin çipe yerleştirilmesi, yani hangi hızda çalışabildiği.

4. Amaç: eğitilmiş bir language model'i FPGA'nin içinde, bilgisayara bağlı olmadan çalıştırmak

Hedefimizi iki düzeni karşılaştırarak anlatmak istiyorum. FPGA hızlandırıcılarında yaygın olan düzen soldaki: bir bilgisayar seri port üzerinden FPGA'ye "şu token'ı işle" der, FPGA bir adım hesaplar, bilgisayar logit'leri okur, bir sonraki token'ı kendisi seçer ve metni kendisi ekrana yazar. Bu düzende FPGA yalnızca bir hesap makinesi; dışarıdan bakıldığında model bilgisayarda çalışıyor gibi görünüyor. Biz bunu bilerek seçmedik. Bizim tasarımımızda modelin weight'leri bitstream ile birlikte çipin BRAM'ine gömülüyor; üretim döngüsü, bir sonraki token'ı seçen sampling, token'ı karakterlere çevirme ve ekrana yazma hepsi çipin içinde. Kart açıldığında model kendiliğinden yazmaya başlıyor ve sonucu doğrudan VGA monitöre veriyor. Üç kural da koyduk: FPGA yalnız inference yapıyor, eğitim Mac'te; yalnız Karpathy'nin modellerini kullanıyoruz; ve doğrulamayı fiziksel kart yerine Mac'te, kartın tamamını simüle ederek yapıyoruz.

5. Hedef kart Nexys A7-100T: işlemcisi olmayan, orta büyüklükte bir Artix-7 FPGA kartı

Hedef kartımız Digilent'in Nexys A7-100T kartı. Üzerinde AMD'nin, eski adıyla Xilinx'in Artix-7 ailesinden XC7A100T çipi var: 63 400 LUT, 126 800 flip-flop, 135 adet 36 kilobitlik BRAM, yani yaklaşık 607 kilobayt çip içi bellek, ve 240 DSP çarpıcı. Kartta ayrı bir işlemci yok; bu da tasarımı "her şey FPGA'de" olmaya zorluyor. Soldaki şekil kullandığımız giriş ve çıkışları gösteriyor: 100 megahertzlik osilatör saatimizi veriyor; VGA çıkışı metni monitöre yazıyor; 7-segment gösterge hızı gösteriyor; 16 anahtar ile sampling modu, temperature ve seed seçiliyor; düğmelerle yeni koşu başlatılıyor, üreteç yeniden tohumlanıyor ve ekran temizleniyor; LED'ler token sayısını ve kart üstü kontrolün sonucunu gösteriyor. Bitstream USB-JTAG ile ya da açılışta QSPI flash'tan yükleniyor. Kartta 128 megabaytlık bir DDR2 bellek de var ama onu bilerek kullanmıyoruz; bunun nedenini bir sonraki slaytta anlatacağım.

6. Tasarımı dört kısıt şekillendirdi: işlemci yok, floating point yok, dış bellek yok, tek clock

Tasarımı dört kısıt şekillendirdi. Birincisi işlemci olmaması: bir bilgisayarda birkaç satır Python olan üretim döngüsü, sampling, token'ı karakterlere çevirme ve ekrana yazma, burada her biri bir donanım durum makinesi olarak yazıldı. İkincisi floating point olmaması: çipteki DSP blokları integer çarpıcılar; floating point toplama ve çarpma LUT'larla kurulabilir ama çok pahalı. Bu yüzden bütün hesabı INT8 ve INT16 ile yaptık; üstel fonksiyon, karekök ve bölme de dahil. Üçüncüsü dış bellek kullanmamak. Bu bir tercih: model çip içine sığdığı sürece bellek gecikmesi olmuyor ve her cycle önceden bilinebiliyor. Sayıyla bakalım: stories260K'nın 260 bin weight'i birer bayt olarak yaklaşık 254 kilobayt; 256 token'lık context için KV cache 5 layer çarpı 256 pozisyon çarpı 32 boyut çarpı key

ve value için 2, yani 80 kilobayt. Toplam 607 kilobaytın rahatça altında. Karşılaştırma için GPT-2'nin 124 milyon parametreli hâli 124 megabayt isterdi; o ancak DDR ile olur. Dördüncüsü tek clock: 100 megahertzlik osilatörden bir MMCM ile çekirdek clock'unu üretiyoruz; VGA'nın 25 megahertzlik piksel hızını da ayrı bir clock yerine aynı clock'tan bir enable sinyaliyle türetiyoruz. Böylece clock domain geçişi problemi hiç yok.

7. Dokuz mimari, Karpathy'nin dört deposundan geliyor ve aynı ~50 bin parametre bütçesiyle eğitildi

Model ailesimiz Karpathy'nin dört deposundan geliyor. İlk beş model makemore'dan: bigram yalnızca son karaktere bakan bir tablo; mlp Bengio'nun 2003'teki modeli gibi son 16 karakterin embedding'lerini birleştiriyor; rnn ve gru recurrent modeller, geçmişi bir state vektöründe taşıyorlar; bow ise geçmişin causal ortalamasını alan basit bir "bag of words". Gri satırlar transformer'lar. gpt2, minGPT ve nanoGPT'nin ortak GPT-2 bloğu: LayerNorm, GELU, öğrenilen pozisyon embedding'leri. microgpt, Karpathy'nin bu yıl yayımladığı 200 satırlık microgpt gist'inden: parametresiz RMSNorm, ReLU, hiç bias yok. llama, llama2.c'nin mimarisi: gain'li RMSNorm, SwiGLU, RoPE ve grouped-query attention. nanochat da Karpathy'nin en yeni deposunun çekirdek mimarisi: parametresiz RMSNorm, ReLU kare, RoPE, GQA ve QK-norm. Karşılaştırma adil olsun diye hepsini aynı veri setinde, names.txt'te, ve aynı yaklaşık 50 bin parametre bütçesiyle eğittik; bigram doğası gereği 729 parametrede kalıyor. Transformer'ların hepsi 4 layer, 32 boyutlu embedding ve 16 token context ile.

8. Mimariler aynı birkaç yapı taşının farklı birleşimleri; engine bu yapı taşlarını birer op olarak sunuyor

Bu dokuz mimari ilk bakışta çok farklı görünüyor, ama donanım açısından aynı birkaç yapı taşının farklı birleşimleri. Bu tablo, her modelin mikrokodunda hangi op türlerinin gerçekten cycle harcadığını gösteriyor; veriler Verilator profilinden. Her model bir embedding okumasıyla başlıyor. bigram için bu tek op yeterli, çünkü embedding satırı doğrudan logit'ler. mlp'den itibaren her modelin ağır işi GEMV, yani matris-vektör çarpımı. recurrent modeller state vektörünü taşımak için COPY ve GRUUPD gibi küçük op'lar kullanıyor; gru ayrıca gate'ler için eleman bazlı çarpma, yani MUL. bow kendi causal ortalama op'unu kullanıyor. Transformer'larda NORM ve attention ile KV cache ekleniyor; llama ve nanochat'te RoPE, llama'da SwiGLU için yine MUL. En sağdaki sütun bir token için mikrokodda kaç op olduğunu gösteriyor: bigram'da 1, nanochat'te 60, stories260K'da 73. Bu tablonun mesajı şu: on altı op türünü destekleyen tek bir engine yazarsak bütün aileyi çalıştırabiliriz. Activation fonksiyonları ayrı bir birim değil; GEMV'nin çıkışında bir tabloyla uygulanıyor, bunu birkaç slayt sonra göstereceğim.

9. stories260K, Karpathy'nin TinyStories'te eğittiği gerçek bir Llama 2; diğer mimarileri de aynı boyda eğittik

Ailenin en önemli üyesi stories260K, çünkü ailede başkasının eğittiği tek model. Karpathy bunu llama2.c deposunda TinyStories veri setinde eğitip yayımladı: küçük çocuk hikâyelerinden oluşan bir veri seti. Mimari tam bir Llama 2: 64 boyutlu embedding, 5 layer, 8 attention head ve 4 KV head, yani grouped-query attention, 512 token'lık bir vocabulary. Context 512 ama kartta KV cache BRAM'e sığsın diye 256 ile sınırlıyoruz. Karpathy'nin checkpoint dosyasını doğrudan okuyoruz ve float hâliyle INT8 hâli arasındaki fark 1,400'den 1,422'ye, yani 0,022 nat. Sağdaki tablo bizim TinyStories

sürümlerimiz: aynı mimarileri aynı veride, stories260K ile aynı boyda, yaklaşık 260 bin parametreyle eğittik. Bu ölçekte Llama mimarisi en iyisi, GPT-2 ikinci; microgpt, gru ve nanochat geride. Burada bir uyarı: sağdaki loss'lar kendi test düzenimizle, stories260K'nınki ise 400 doğrulama hikâyesiyle ölçüldü; bu yüzden iki tablo arasında doğrudan karşılaştırma yapmıyorum.

10. Kartın ekranından: aynı devre bir modelle isim, diğeriyle hikâye yazıyor

Bunlar kartın VGA çıkışından iki kare. Fotoğraf değil: kartın bütününü simüle ederken VGA pinlerinden, yani kırmızı, yeşil, mavi ve senkron sinyallerinden yakaladığımız kareler. Python'daki referans ekranla karşılaştırdığımızda tek bir piksel bile farklı değil. Soldaki gpt2 modeli: names.txt'te eğitildiği için alt alta yeni isimler uyduruyor; bu koşuda 196 token, token başına 8 713 cycle. Sağdaki Karpathy'nin stories260K modeli: "Once upon a time, there was a little girl named Lily" diye başlayan 256 token'lık bir hikâye yazıyor; token başına 50 397 cycle. Üstteki satır model başlığı, alttaki durum satırı token sayısını, hızı, cycle sayısını, sampling ayarlarını ve kart üstü kontrolün sonucunu gösteriyor: "check OK", yani kart Python ile aynı token'ları üretmiş. Durum satırındaki token/s değeri 50 megahertz varsayımıyla hesaplanıyor; yerleşim bölümünde bu clock'un gerçekte nereye indiğini göstereceğim. İki ekranı da aynı devre üretiyor; değişen yalnızca BRAM'deki model.

11. Neden integer: aynı çarpma-toplamayı bir DSP tek cycle'da yapıyor ve model dört kat küçülüyor

Neden integer? İki sebep var. Birincisi hesap. Bir float32 çarpma-toplama dört adım istiyor: üsleri toplayıp mantisleri çarpmak, toplanacak sayıyla üsleri hizalamak için kaydırmak, toplayıp sonucu normalize etmek ve yuvarlamak. FPGA'de hazır bir floating point birimi yok; bunların hepsi LUT'larla kurulmalı ve hem pahalı hem yavaş. INT8'de ise ortadaki örnekteki gibi: x ve w vektörlerinin dot product'ı 12 çarpı 18, eksi 7 çarpı eksi 55 ve 30 çarpı 127; toplam 4 411. Her terim bir DSP48 bloğunda tek cycle'da yapılıyor: 8 bitlik iki sayıyı çarp, 32 bitlik bir akümülatöre ekle. İkinci sebep bellek. stories260K'nın weight'leri float32 olarak bir megabaytın üzerinde ve çipin 607 kilobaytlık BRAM'ine sığmıyor bile. INT8'de 254 kilobayt; rahatça sığıyor. Bunun bir bedeli var: her sayı artık bir scale ile çarpılmış bir tamsayı. Asıl soru, bu scale'leri nasıl seçtiğimiz ve tamsayılar arasındaki işlemleri hiç float'a dönmeden nasıl yaptığımız. Sonraki beş slayt bunun cevabı.

12. Quantization: her gerçel sayı, bir scale ile çarpılmış bir tamsayı olarak yazılır

Quantization'ın fikri çok basit: her gerçel sayıyı bir scale ile çarpılmış bir tamsayı olarak yazıyoruz. Soldaki örneğe bakalım. Bir weight satırında mutlak değeri en büyük eleman 0,268 olsun. INT8'in pozitif tarafı 127'ye kadar gidiyor, o yüzden scale S, 0,268 bölü 127, yani 0,00211. Şimdi her weight'i S'ye bölüp yuvarlıyoruz. 0,0372, 17,63 ediyor ve 18'e yuvarlanıyor; geri çevirdiğimizde 0,0380 elde ediyoruz, hata 0,0008. Eksi 0,1153, eksi 55 oluyor. En büyük eleman tam 127'ye denk geliyor ve hatası sıfır. Çok küçük 0,0009 ise 0,43 ediyor ve sıfıra yuvarlanıyor. Hata hiçbir zaman S'nin yarısını, yani 0,00106'yı geçmiyor. Sağdaki formül bunun genel hâli. Önemli iki tasarım kararı var. Weight'ler için her çıkış kanalının, yani her satırın kendi scale'i var; böylece büyük değerli bir satır küçük değerli bir satırın çözünürlüğünü bozmuyor. Activation'lar için ise tensör başına sabit bir scale kullanıyoruz ve bunu kalibrasyon verisinden, mutlak değerin yüzde 99,99'luk yüzdeliğinden

seçiyoruz. Maksimum yerine yüzdelik seçmek, nadir uç değerleri kırpıp geri kalan değerlere daha ince bir çözünürlük kazandırıyor; bunu model ailesi üzerinde ölçerek seçtik.

13. Requant: gerçel çarpan M, bir tamsayı çarpan ve bir kaydırma olur; float'a hiç dönülmez

Asıl numara requant. Bir Linear katmanın çıkışını düşünelim. Weight'ler S_w, girişler S_x scale'inde tamsayılar. Tamsayıları çarpıp topladığımızda bir akümülatör elde ediyoruz ve gerçek çıkış yaklaşık olarak S_x çarpı S_w çarpı bu akümülatör. Ama bir sonraki katman bu çıkışı kendi scale'i S_y'de INT8 olarak bekliyor. Yani akümülatörü M ile çarpıp yuvarlamamız gerekiyor; M de S_x çarpı S_w bölü S_y, bir gerçel sayı. Float kullanmadan bunu nasıl yaparız? M'yi bir tamsayı m_0 ile 2 üzeri eksi sh'nin çarpımı olarak yazıyoruz; m_0'ı 2 üzeri 15 ile 2 üzeri 16 arasında seçince bağıl hata 2 üzeri eksi 16'dan küçük kalıyor. Donanımda bu bir çarpma, yarımın eklenmesi ve bir sağa kaydırma: yuvarlamayı kaydırmadan önce 2 üzeri sh eksi 1 ekleyerek yapıyoruz. Sağda sayısal bir örnek var: scale'ler 0,05, 0,00211 ve 0,077 olsun. M, 0,0013703 çıkıyor; sh 25 seçilince m_0 45 979 oluyor. Akümülatör 5 321 ise çarpım 244 654 259, yarımı ekleyince 261 431 475, 25 bit kaydırınca 7. Float ile hesaplasaydık 5 321 çarpı M, 7,29 ve yuvarlayınca yine 7. Bu yöntem Google'ın 2018'deki integer-only inference makalesinden; bizim tasarımda her çıkış kanalının m_0 ve sh değerleri parametre ROM'unda duruyor.

14. Integer softmax: üstel fonksiyon bir kaydırma ile 256 satırlık bir tabloya bölünür, bölme bir kez yapılır

Attention'ın kalbindeki softmax'ı integer'da nasıl hesaplıyoruz? Üç adım var. Birincisi: en büyük score'u bulup her score'dan çıkarıyoruz. Softmax bundan etkilenmiyor ama artık bütün üsler sıfır ya da negatif, yani hiçbir değer taşmıyor. Fark, scale'leri ve $\log_2 e$'yi içeren bir sabit C ile çarpılıyor. İkincisi: e üzeri x'i 2 üzeri x çarpı $\log_2 e$ olarak yazıyoruz. Bu üssün tam kısmı n, 2 üzeri eksi n ile çarpmak demek, bu da basit bir sağa kaydırma. Kesirli kısım f ise 256 satırlık bir tablodan okunuyor. Üçüncüsü: olasılıkları tek tek bölmüyoruz; önce bütün e_i'leri ve value'larla ağırlıklı toplamı biriktiriyoruz, en sonda bir kez 2 üzeri 32 bölü E'yi hesaplayıp çarpıyoruz. Yani her head için tek bir bölme. Sağdaki örnek: score'lar 9, 6 ve 2; C 92. Fark çarpı C sırasıyla 0, 276 ve 644. 276, 256 çarpı 1 artı 20 olarak ayrışıyor: tablonun 20. satırını okuyup bir bit kaydırıyoruz ve 15 520 buluyoruz. Toplam 54 018; bölünce olasılıklar 0,607, 0,287 ve 0,106. Float softmax da üç basamakta aynı sonucu veriyor. Aynı tablo, kartta bir sonraki token'ı seçen sampler'da da kullanılıyor.

15. Integer LayerNorm ve RMSNorm: vektör başına bir karekök ve bir bölme; çıkışın taşmayacağı ispatlı

Normalization katmanları, yani LayerNorm ve RMSNorm, ortalama, varyans, karekök ve bölme istiyor. Bunları integer'da beş aşamada yapıyoruz. Önce elemanları topluyoruz. Sonra her elemanı N ile çarpıp toplamı çıkarıyoruz: bu, N çarpı x eksi ortalama demek, ama bölme yok ve hiçbir bilgi kaybolmuyor. Bu farkların karelerini toplayıp epsilon'un karşılığı olan sabiti ekliyoruz. Sonra bu toplamın tamsayı karekökünü bit bit, sıralı bir devreyle alıyoruz; sonra sabit bir K'yı bu köke bölüyoruz. Son aşamada her farkı bu tersle çarpıp kaydırıyoruz. RMSNorm'da ortalama çıkarılmıyor; aynı birimde toplamı sıfır alıyoruz. Bu tasarımın güzel bir garantisi var: Samuelson eşitsizliğine göre her elemanın ortalamadan uzaklığı en fazla sigma çarpı kök N eksi 1. Yani normalize edilmiş çıkışın

büyüklüğü veriden bağımsız olarak sınırlı; çıkış formatını bu sınıra göre seçince ne kalibrasyon gerekiyor ne de doyma oluyor. Üçüncü nokta: LayerNorm'un gain'i ve bias'ı, arkasından gelen Linear'ın weight'lerine ve bias'ına matematiksel olarak katlanıyor; donanım yalnız normalize edilmiş vektörü hesaplıyor. Maliyete bakarsak NORM, gpt2'de cycle'ların yüzde 14'ü; nanochat'te QK-norm yüzünden yüzde 33'ü.

16. INT8 girişli her fonksiyon 256 satırlık bir tablodur: GELU, SiLU, tanh, σ ve ReLU² yaklaşımsız

Activation fonksiyonları için çok basit ama güçlü bir gözlem kullanıyoruz: bir fonksiyonun girişi INT8 ise giriş yalnızca 256 farklı değer alabilir. O hâlde fonksiyonu 256 satırlık bir tabloyla tam olarak temsil edebiliriz; polinom yaklaşımına ya da parça parça doğrusal bir yaklaşıma gerek yok. Soldaki grafik GELU için: mavi çizgi gerçek GELU, turuncu noktalar tablonun değerleri. Noktalar eğrinin tam üzerinde; tek fark, çıkış 127'de doyuyor. Sağdaki tablo birkaç satır: giriş tamsayısı 20 ise gerçek değer 1,0, GELU'su 0,841 ve çıkış scale'ine bölünüp yuvarlanınca 34. Bu yöntemle GELU, SiLU, tanh, sigmoid ve ReLU kare, hepsi aynı mekanizmayla yapılıyor; her model hangi fonksiyonu kullanıyorsa derleyici o tabloyu üretiyor. Engine'de ayrı bir activation birimi yok: tablo GEMV'nin çıkışına takılıyor; akümülatör önce requant ile INT8'e iniyor, sonra tablodan geçiyor. Katman başına 256 bayt.

17. Bedel küçük: INT8'e geçmek names modellerinde en fazla 0,006, stories260K'da 0,022 nat kaybettiriyor

Bütün bu integer hilelerinin kaliteye bedeli ne? Bu tablo, float model ile donanımla bit bit aynı çalışan integer programın aynı test verisindeki loss'larını karşılaştırıyor. Çubuklar loss farkı. names modellerinde fark en fazla 0,0055 nat, nanochat'te; bigram ve bow'da pratikte sıfır, hatta bow'da integer model binde üç daha iyi, bu gürültü düzeyinde. En büyük fark Karpathy'nin stories260K modelinde: 0,022 nat. Bu beklenen bir şey, çünkü bu model bizim kalibrasyonumuzu bilmeden, float olarak eğitilmiş ve 5 layer derinliğinde; quantization hataları layer'lar boyunca birikiyor. Yine de 1,400'e karşı 1,422, yüzde 1,5'lik bir fark. Son sütun başka bir bakış: her adımda en olası token'ın float ile integer modelde aynı çıkma oranı. Küçük modellerde yüzde 96 ile 100 arası, stories260K'da yüzde 92. Bu farklılıklar genellikle olasılıkları birbirine çok yakın iki token arasında oluyor; loss farkının bu kadar küçük olmasının sebebi bu.

18. Derleyici, bir PyTorch modelini altı adımda donanımın okuyacağı mikrokoda ve bellek imajlarına çevirir

Bir modeli donanıma götüren yazılım tarafı, yani derleyici, altı adımdan oluşuyor. Birinci adımda eğitilmiş modeli alıyoruz: kendi eğittiklerimiz için PyTorch dosyası, stories260K için Karpathy'nin llama2.c formatındaki .bin dosyası. İkinci adımda modeli bir ara temsile, IR'a çeviriyoruz: embedding, norm, GEMV, attention gibi op'ların düz bir listesi. Bu listeyi float olarak çalıştırıp PyTorch modeliyle karşılaştırıyoruz; fark 10 üzeri eksi 4'ün altında olmalı. Üçüncü adımda matematiksel olarak eşdeğer dönüşümleri yapıyoruz: norm'un gain ve bias'ını, attention'daki 1 bölü kök d'yi weight'lere katlıyoruz. Dördüncü adım kalibrasyon: örnek verilerle activation'ların scale'lerini seçiyoruz. Beşinci adımda integer programı elde ediyoruz. Bunu çalıştıran Python sınıfı IntRunner bizim altın referansımız: donanım, bu programın ürettiği her sayıyı bit bit aynı üretmek zorunda. Altıncı adımda programı ve verileri, Verilog'un readmemh ile okuyacağı bellek dosyalarına

yazıyoruz; bitstream üretilirken bunlar BRAM'lerin başlangıç içeriği oluyor. Tablo gpt2 için dosyaları gösteriyor: 43 op'luk mikrokod, 64 bitlik 6 272 weight kelimesi, kanal başına requant parametreleri ve tablolar.

19. Mikrokod, modelin forward fonksiyonunun op op yazılmış hâli: gpt2 ile llama'nın bir block'u

Mikrokodun neye benzediğini görmek için iki modelin ilk block'unu yan yana koydum. Bunlar derleyicinin gerçekten ürettiği op'lar. Soldaki gpt2: önce residual tampon R'yi LayerNorm ile normalize edip A'ya yazıyoruz; A'dan üç GEMV ile query, key ve value'yu hesaplıyoruz; key ve value'yu KV cache'e yazıyoruz; attention dört head için çıkışı B'ye yazıyor; projection GEMV'si sonucu R'ye ekliyor, yani residual bağlantı "artı eşittir" modunda bir GEMV. Sonra ikinci LayerNorm, GELU tablosu çıkışa takılı fc GEMV'si ve yine residual'a eklenen bir projection. Sağdaki llama'da yapı aynı, üç fark turuncu: query ve key'e RoPE uygulanıyor, key için yalnız 2 head var çünkü grouped-query attention; ve MLP SwiGLU: w1'in çıkışı SiLU tablosundan geçiyor, w3'ün çıkışıyla eleman bazında çarpılıyor, sonra w2. Önemli olan şu: iki mimari arasındaki fark birkaç op; geri kalan her şey aynı ve engine ikisini de aynı devreyle çalıştırıyor. Programın sonunda son norm ve logit'leri üreten head GEMV'si var.

20. En büyük model stories260K bile BRAM'in yaklaşık %71'ini kullanıyor; en büyük pay weight'lerin

Modelin çipin içine sığması tasarımın temel varsayımıydı; bunu en büyük modelle kontrol edelim. Çubuk, stories260K'nın çip içi belleğini gösteriyor; çubuğun tamamı 607 kilobayt. En büyük pay weight'lerin: 256 kilobayt. Sonra KV cache, 256 pozisyonluk context için 80 kilobayt. Token embedding'leri ve exp tablosu gibi tablolar 68 kilobayt; her çıkış kanalının bias, m_0 ve sh değerleri 28 kilobayt; mikrokod ve activation tabloları birkaç kilobayt. Geriye 170 kilobayt kadar boş yer kalıyor. Yosys sentezi bunu 97,5 BRAM bloğu, yani kapasitenin yüzde 72'si olarak tahmin ediyor. Alttaki tablo karşılaştırma: names modelleri yüzde 1 ile 14 arasında; binary stories260K ise weight'ler 8 kat küçüldüğü için yüzde 41'e iniyor. Önemli bir ayrıntı: weight'ler bitstream'in bir parçası. Verilog'daki readmemh ile okunan .mem dosyaları BRAM'lerin başlangıç içeriği oluyor; kart açılırken FPGA yapılandırılırken weight'ler de yerine yazılmış oluyor. KV cache ve vektör tamponları ise çalışırken yazılan sıradan RAM'ler.

21. zoo_core: mikrokodu op op okuyan, her op'u kendi birimine gönderen tek bir engine

Bütün modelleri çalıştıran engine'in adı zoo_core. Çalışma şekli bir işlemciye benziyor ama çok daha basit. Üstteki şerit kontrol akışı: engine program belleğinden bir op kelimesi okuyor; bu kelimede op türü, kaynak ve hedef tamponlar, vektör boyutları ve weight adresleri var. Op'u ilgili birime gönderiyor, birim bitince bir sonraki kelimeye geçiyor; END op'una gelince bir token'ın hesabı bitmiş oluyor ve logit'ler üretim döngüsüne gidiyor. Soldaki sütun modele ait bellekler: INT8 weight'ler, 1 bitlik weight'ler, requant parametreleri, tablolar ve çalışırken yazılan KV cache. Ortada hesap birimleri. Turuncu olanlar matris-vektör çarpımı: 8 hatlı INT8 GEMV, 64 hatlı 1-bit GEMVB ve ikisinin ortak kullandığı seri requant ve activation tablosu. Mavi olanlar transformer'a özgü birimler: norm, attention ile KV cache yazımı ve RoPE. Alttakiler küçük yardımcı op'lar: embedding okuma, eleman

bazlı çarpma ve recurrent modellerin state işlemleri. Sağda 11 vektör tamponu var; residual akış INT16, diğerleri INT8. Hepsi tek bir modülde; bir model hangi op'ları kullanmıyorsa o birimler sentezde atılabiliyor, bunu birkaç slayt sonra göstereceğim.

22. GEMV: her cycle'da bir giriş 8 hatta yayınlanır ve 8 çıkış aynı anda birikir

Hesabın büyük kısmını yapan GEMV birimine yakından bakalım. Soldaki şekil bir weight matrisinin 8 satırlık bir çıkış bloğunu ve giriş vektörünü gösteriyor. Her cycle'da giriş vektörünün bir elemanı, mesela x_3, 8 hattın hepsine birden yayınlanıyor. Aynı cycle'da weight belleğinden 64 bitlik tek bir kelime okunuyor; bu kelimede o girişe ait 8 farklı çıkışın weight'i yan yana duruyor. Her hat kendi weight'ini x ile çarpıp kendi akümülatörüne ekliyor. Giriş vektörü bitince 8 çıkışın akümülatörü hazır. Bu düzenin adı output-stationary: çıkışlar yerinde kalıyor, girişler akıyor. Weight belleği tam bu sırayla yazıldığı için adres basit bir sayaç. Blok bitince 8 akümülatör bir gölge register bankasına kopyalanıyor ve oradan requant birimine tek tek akıyor; bu sırada hatlar bir sonraki 8 çıkışa başlamış oluyor, yani boşluk yok. Cycle sayısı bu yüzden basit: çıkış sayısının 8'e bölümü çarpı giriş sayısı. gpt2'nin 32'den 128'e fc katmanı 16 blok çarpı 32, 512 cycle; stories260K'nın 64'ten 172'ye w1 katmanı 22 blok çarpı 64, 1 408 cycle. Bu yüzden bir modelin token başına cycle sayısı önceden, kağıt üzerinde hesaplanabiliyor.

23. Attention birimi: her geçmiş token, her head'de iki cycle ekler; bu yüzden maliyet context ile doğrusal büyür

Attention birimi dört aşamalı. SCORE aşamasında query, KV cache'teki her key ile çarpılıyor; head boyutu 8 olduğu için 8 çarpıcı bir key'in dot product'ını tek cycle'da bitiriyor. Score'lar gelirken EXP aşaması onları softmax için 2'nin kuvvetine çeviriyor ve topluyor; bu ikisi örtüşüyor. DIV aşamasında head başına tek bir bölme yapılıyor. AV aşamasında value'lar ağırlıklarıyla toplanıyor, yine 8 hatla. Sonuç: her geçmiş token, her head'de score için bir, value için bir, yani iki cycle ekliyor. Bunu ölçümle doğrulayabiliriz. stories260K'da ilk pozisyonda token başına 38 652 cycle, son pozisyon 511'de 79 532 cycle. Aradaki fark bölü 511, tam 80,0 ediyor; model de 5 layer çarpı 8 head çarpı 2, yani 80 diyor. Ölçüm ile model birebir aynı. Bu doğrusal büyüme, kartta context'i neden 256 ile sınırladığımızın ikinci sebebi: hem KV cache 80 kilobaytta kalıyor hem de en uzun adım yaklaşık 59 bin cycle'da kalıyor.

24. Cycle'lar nereye gidiyor: makemore'da neredeyse hepsi GEMV; transformer'larda norm ve attention da pay alıyor

Engine'in cycle'ları nereye harcadığına bakalım; veriler Verilator'daki op bazlı profilden. Turuncu GEMV, koyu mavi norm, açık mavi attention, gri diğer op'lar. makemore modellerinde, yani mlp, rnn, gru ve bow'da, cycle'ların yüzde 94 ile 96'sı GEMV; bunlar neredeyse saf matris-vektör çarpımı. Transformer'larda tablo değişiyor: gpt2'de GEMV yüzde 77, norm yüzde 14, attention yüzde 7. nanochat'te norm yüzde 33'e çıkıyor, çünkü nanochat her query ve key'i de normalize ediyor, QK-norm; bu da her layer'da ek norm demek. stories260K'da ise attention yüzde 37 ile ikinci büyük kalem; çünkü burada context 512'ye kadar çıkıyor ve attention önceki slayttaki gibi pozisyonla büyüyor. Bu profil, optimizasyonun nereye yapılması gerektiğini söylüyor: küçük modellerde GEMV paralelliği, uzun context'te attention, nanochat'te norm. Sağdaki sayılar engine'in token başına ortalama cycle sayısı; 9 ile 59 bin arasında.

25. SPECIALIZE: modelin kullanmadığı birimler sentezde atılır; makemore modellerinde engine dörtte birine iner

Engine bütün op'ları desteklese de her model hepsini kullanmıyor; bigram yalnızca bir tablo okuyor. Bu yüzden bir SPECIALIZE parametresi ekledik. Derleyici, her model için hangi op'ların kullanıldığını bir bit maskesine yazıyor; sağdaki kod parçasındaki OPS_USED bu. SPECIALIZE açıkken her birimin mantığı bu maskenin ilgili bitiyle koşullanıyor; bit sıfırsa o mantık sabit olarak kapalı oluyor ve sentez aracı onu tamamen siliyor. Soldaki çubuklar bunun etkisi: gri, bütün birimlerle; turuncu, yalnızca kullanılan birimlerle LUT sayısı. bigram 5 855'ten 1 529'a, mlp 8 094'ten 2 502'ye iniyor; makemore modellerinde engine yaklaşık dörtte birine küçülüyor. Transformer'lar zaten çoğu birimi kullandığı için kazanç az: stories260K'da yüzde 9. Kart tasarımında her model kendi uzmanlaşmış engine'iyle sentezleniyor. Karşılaştırma için bütün birimleri içeren ve modeli sonradan yükleyebilen tek bitstream'lik sürüm 9 444 LUT.

26. Engine'in etrafında, bir bilgisayarın yapacağı her işi yapan donanım: döngü, sampling, metin ve ekran

Engine tek başına yalnızca "bu token ve bu pozisyon için logit'ler" sorusunu cevaplıyor. Bir language model'i kullanılabilir yapan geri kalan her iş, normalde bir bilgisayarda Python ile yapılıyor. Biz bunları da donanıma koyduk. Üst sıra kontrol yolu: anahtar ve düğmeler debounce edilip senkronlanıyor; üretim döngüsü engine'e token ve pozisyonu veriyor, logit'leri alıp sampler'a gönderiyor ve seçilen token'ı bir sonraki adımın girişi yapıyor. Alt sıra çıkış yolu: detokenizer token numarasını karakterlere çeviriyor, stories260K'da bir token birden fazla karakter olabiliyor; terminal bu karakterleri 80 sütunluk bir metin alanına yazıyor, kelimeleri bölmeden alt satıra kaydırıyor, ekran dolunca scroll ediyor ve en alttaki durum satırını güncelliyor; vga_text de bu metni 640 çarpı 480 çözünürlükte, 8'e 16 piksellik bir fontla monitöre çiziyor. Altta üç ayrıntı: kart üstü kontrol, ekran yazımının bir sonraki token'ın hesabıyla paralel yürümesi, yani maliyetsiz olması, ve 7-segment ile LED'ler.

27. Üretim döngüsü bir durum makinesi: engine'i çalıştırır, logit'leri üç kez tarar ve bir token seçer

Üretim döngüsü bir durum makinesi. G_STEP'te engine'e mevcut token'ı ve pozisyonu verip başlatıyor, G_WAIT'te engine'in END op'una gelmesini bekliyor. Sonra logit'leri üç kez tarıyor; turuncu satırlar bunlar. G_MAX en büyük logit'i buluyor; greedy moddaysak seçim bu kadar, en büyük logit'in token'ı doğrudan seçiliyor. Sampling modunda G_SUM her token için 2 üzeri eksi fark çarpı c'yi hesaplıyor, ki bu temperature'lı softmax'ın normalize edilmemiş hâli; aynı anda min-p kesmesini uyguluyor ve toplamı biriktiriyor. G_SEL'de rastgele bir sayıyı toplamla çarpıp bir eşik buluyor ve birikimli toplam bu eşiği ilk aştığında o token'ı seçiyor. G_PICK seçilen token'ı ekrana gönderiyor, hash'i güncelliyor, pozisyonu artırıyor ve döngü başa dönüyor. Bu taramaların her biri vocabulary boyu kadar cycle sürüyor; toplam ek maliyet yaklaşık 3V. stories260K'da bu 1 500 cycle civarı, token başına toplamın yüzde 3'ü; names modellerinde vocabulary 27 olduğu için 80 cycle kadar.

28. Integer sampling: temperature ve min-p, softmax ile aynı tabloyla ve hiç bölme yapmadan

Bir sonraki token'ı seçmek, yani sampling, normalde temperature ile bölünmüş logit'lerin softmax'ını alıp o dağılımdan rastgele bir örnek çekmek demek. Bunu da integer'a çevirdik. Logit'lerin en büyüğünü çıkarıp farkı c ile çarpıyoruz; c, logit scale'ini, $\log_2 e$'yi ve temperature'ı içeren bir sabit. Sonra attention'daki aynı 2'nin kuvveti tablosuyla e_k'leri buluyoruz. Min-p kuralı, olasılığı en olası token'ın 32'de birinden küçük olanları atıyor; en büyük e her zaman 2 üzeri 15 olduğu için bu basit bir karşılaştırma. Bölmeye hiç gerek yok: olasılıkları normalize etmek yerine rastgele sayıyı toplamla çarpıp 32 bit kaydırıyoruz ve birikimli toplamın bu eşiği ilk aştığı token'ı seçiyoruz. Sağdaki örnek dört token'lı: logit'ler 120, 100, 60 ve 20; scale 0,05, temperature 0,8. Üçüncü ve dördüncü token min-p ile kesiliyor. Kalan iki token'ın olasılıkları 0,7775 ve 0,2225; float hesap 0,7773 ve 0,2227 veriyor, binde birden küçük fark. Rastgele sayı 0,75'e karşılık geliyorsa eşik 31 610 ve ilk token seçiliyor; 0,875'e karşılık geliyorsa ikinci token. Python'daki referans aynı algoritmayı birebir uyguladığı için kartın seçtiği token'lar Python'unkilerle aynı.

29. Ekran: token'lar karakterlere çevrilir, kelime bölünmeden yazılır ve VGA'ya her karede yeniden çizilir

Ekran tarafında üç iş var. Detokenizer, token numarasını karakterlere çeviriyor. Bunun için iki ROM kullanıyoruz: biri her token'ın karakter dizisinin nerede başladığını ve kaç karakter olduğunu, diğeri karakterlerin kendisini tutuyor. names modellerinde her token bir harf; stories260K'da bir token " the" gibi birkaç karakter olabiliyor. Terminal bu karakterleri 80 sütunluk bir metin alanına yazıyor. İlk sürümde kelimeler satır sonunda ortadan bölünüyordu; şimdi bir kelime 80. sütunu aşacaksa yazılmış yarısı alt satıra taşınıyor. Şekilde satır sonlarına bakarsanız hiçbir kelimenin bölünmediğini görürsünüz. Ekran dolunca satırlar dairesel bir tamponda kayıyor. vga_text, 640 çarpı 480 çözünürlükte, saniyede 60 kare, her karede metin RAM'ini baştan okuyup font ROM'undan pikselleri çiziyor; üç kademeli bir pipeline. En alttaki durum satırındaki token/s değerini bir bölücüyle hesaplıyor ve sayıları bin2bcd modülüyle onluğa çeviriyoruz. Bu kare de pinlerden yakalandı ve referansla piksel piksel aynı.

30. Kullanım: anahtarlar üretimi ayarlar, düğmeler koşu başlatır; LED16 kartın Python ile aynı sonucu bulduğunu söyler

Kart bir bilgisayara bağlı olmadığı için bütün kontrol anahtar ve düğmelerden. Güç verildiğinde ya da reset'e basıldığında ilk koşu kendiliğinden başlıyor: stories260K bir hikâye, names modelleri 27 isim yazıyor. Orta düğme yeni bir koşu başlatıyor; rastgele sayı üretecinin durumu kaldığı yerden sürdüğü için her basışta farklı bir hikâye geliyor. Üst düğme üreteci anahtarlardan okunan seed ile yeniden tohumluyor, alt düğme ekranı temizliyor. Anahtarlarla greedy ya da sampling seçiliyor, sekiz temperature ve dört min-p değeri arasından seçim yapılıyor; daktilo modu üretimi gözle izlenebilecek bir hıza indiriyor; 7-segment gösterge token/s ya da token başına cycle gösteriyor. Sağdaki kutu kart üstü kontrol: kart ürettiği her token ile basit bir hash güncelliyor. Python, modeli dışa aktarırken aynı koşuların hash'ini hesaplayıp bitstream'e gömüyor; kart koşu bitince ikisini karşılaştırıyor. RGB LED üretim sırasında mavi, sonuç aynıysa yeşil, farklıysa kırmızı yanıyor. Yani kart, Python ile token token aynı sonucu bulduğunu kendisi söylüyor; simülasyonda 18 modelin hepsinde yeşil.

31. 1 bitlik weight, belleği 8 kat küçültür ve çarpmayı toplama-çıkarmaya çevirir

Bu bölüm, hocamın önerdiği iki kaynağın, LittleBit makalesinin ve Larq kütüphanesinin, projede nasıl kullanıldığını anlatıyor. Önce motivasyon. Soldaki INT8 örneği daha önce gördüğümüz dot product: üç çarpma. Sağda aynı weight'leri yalnızca işaretleriyle ve tek bir scale ile temsil ediyoruz: artı, eksi, artı. Artı bir ile çarpmak sayıyı olduğu gibi almak, eksi bir ile çarpmak işaretini çevirmek demek. Yani 12 artı 7 artı 30, 49; sonra bir kez scale ile çarpıyoruz. Matris-vektör çarpımının ağır kısmında hiç çarpma kalmıyor ve her weight 8 yerine 1 bit. LittleBit bunu bir adım ileri götürüp weight matrisini low-rank binary factor'lara ayırarak weight başına 1 bitin altına iniyor; Larq ise binary ağlar için bir kütüphane ve Bi-Real Net, ternary weight networks gibi bilinen quantizer'ları içeriyor. Kural değişmedi: FPGA'de eğitim yok. Düşük bitli weight'leri Mac'te quantization-aware training ile öğreniyoruz, FPGA yalnızca sonucu çalıştırıyor. Literatürdeki gibi embedding ve output head INT8 kalıyor.

32. Beş yöntemi aynı eğitim düzeniyle denedik: Larq'ın quantizer'larını ve LittleBit'i PyTorch'ta yeniden yazarak

Beş yöntemi aynı eğitim düzeniyle karşılaştırdık. int8 engine'in bugünkü formatı ve eğitim gerektirmiyor. int4, standart quantization-aware training ile 4 bit. Ternary ve binary Larq'ın quantizer'ları: ternary, Larq'ın ternary weight networks ayarındaki SteTern'ü, weight'leri eksi a, sıfır ve artı a'ya indiriyor. Binary, Larq'ın Bi-Real Net'ten gelen MagnitudeAwareSign'ı ile her weight'i işaretine indiriyor ve kanal başına ortalama mutlak değeri scale olarak kullanıyor. Turuncu son satır LittleBit: weight matrisi, iki binary factor ve üç scale vektörünün çarpımı olarak temsil ediliyor; başlangıç makaledeki Dual-SVID yöntemiyle, rank ve yol sayısı ayarlanabiliyor. Bu yöntemlerin hepsinin eğitimindeki ortak hile straight-through estimator: ileri yönde weight'in işareti kullanılıyor, geri yönde ise gradient, işaret fonksiyonu yokmuş gibi, ama yalnızca weight eksi bir ile artı bir arasındaysa geçiriliyor. Eğitim gerçek değerli bir gölge weight'i güncelliyor. Ayrıca float modeli öğretmen yapıp knowledge distillation kullandık. Larq aslında bir TensorFlow kütüphanesi; quantizer'ları birkaç satırlık fonksiyonlar olduğu için onları belgelenen tanımlarına göre PyTorch'ta yeniden yazdık. Böylece modellerimiz ve integer akışımız hiç değişmedi.

33. Sonuç: transformer'da binary, geniş girişli MLP'de LittleBit daha iyi; INT4 her yerde float kadar iyi

İki modelin sonuçları üzerinden anlatayım; diğer altı model aynı örüntüyü gösteriyor. Mavi noktalar gpt2, turuncular mlp; yatay eksen QAT sonrası test loss, yani solda olan daha iyi; sağdaki sayılar weight başına bit. İlk gözlem: INT4 her iki modelde de float kadar iyi, gpt2'de knowledge distillation sayesinde biraz daha bile iyi. İkinci gözlem: ternary binary'den fazla bit harcıyor ama binary'den iyi değil. Asıl ilginç olan binary ile LittleBit karşılaştırması. gpt2'de binary 1,38 BPW'de 2,029 ile float'a çok yakın; LittleBit aynı ya da daha fazla bitte daha kötü, mesela lb25p2 2,08 BPW'de 2,090. mlp'de durum tersine dönüyor: binary 2,160 ile kötü, LittleBit lb25p2 1,18 BPW'de 2,075 ile çok daha iyi. Fark katmanların genişliğinden geliyor: gpt2'nin katmanları 32 genişliğinde, mlp'nin ilk katmanının girişi ise 256. Bunun neden böyle olduğunu bir sonraki slaytta formülle göstereceğim. Bu bulgu hocamın verdiği araştırma sorusunun cevaplarından biri: LittleBit küçük modellerde ancak geniş katmanlarda işe yarıyor.

34. Neden: LittleBit'in scale vektörleri dar katmanlarda weight başına bitin büyük kısmını yiyor

Bir önceki slayttaki örüntünün sebebi bu formülde. LittleBit bir katman için iki tür bit harcıyor: binary factor'ların işaret bitleri ve 16 bitlik scale vektörleri, yani h, g ve ℓ. İlk terim rank ile, ikinci terim katmanın boyutlarıyla orantılı. Weight sayısına böldüğümüzde scale'lerin bedeli katman genişliği d ile bir bölü d olarak azalıyor. Sağdaki tablo bunu sayılarla gösteriyor. names modellerindeki 32'ye 32'lik bir katmanda, hiç işaret biti saymadan, yalnızca scale'ler weight başına 1 bit tutuyor; bu, binary bir katmanın toplam 1,5 bitlik bedelinin üçte ikisi. Yani LittleBit'in bit bütçesinin çoğu rank'a değil scale'lere gidiyor; binary ise aynı bütçeyle tam rank'lı bir işaret matrisi kullanıyor. 4096'ya 4096'lık bir LLM katmanında ise yalnız scale'ler 0,008 BPW; tamamen ihmal edilebilir. Makaledeki 0,1 BPW sonucu tam olarak bu genişlikten geliyor: 4096 kare, rank 86, iki yol, 0,0998 BPW. Hocamın sorduğu ilk araştırma sorusunun cevabı bu: küçük modellerde LittleBit'in anlamlı alt sınırını katman genişliği belirliyor ve 64'ün altında binary daha verimli.

35. Donanım kararı: engine'e 64 hatlı, çarpıcısız bir 1-bit GEMV birimi; binary ve LittleBit aynı birimde çalışıyor

Sonuçlara göre donanım kararı şu oldu: engine'e 1 bitlik weight'lerle çalışan bir matris-vektör birimi ekledik, adı GEMVB. 64 bitlik bir weight kelimesi, bir girişin 64 farklı çıkış için işaretlerini taşıyor; 64 hat aynı anda girişi ya ekliyor ya çıkarıyor. Çarpıcı yok, DSP kullanılmıyor. Akümülatörler 20 bit yetiyor, çünkü her terim en fazla 128. Requant ve kontrol mantığı INT8 GEMV ile ortak; kanal başına scale de requant çarpanına katlanıyor. İkinci sorun normalization'ın gain'iydi. Derleyici normalde γ'yı arkadaki weight'e katlıyor; ama binary bir matris çarpı diag γ artık binary değil. Bu yüzden katlanmış matrisi satır scale'i, işaret matrisi ve sütun scale'i olarak ayırıyoruz. Satır scale'i requant'a gidiyor; sütun scale'i ise yeni bir SCALE op'u oluyor ve aynı girişi okuyan q, k ve v için tek seferde uygulanıyor. Üçüncü kutu: tek yollu LittleBit de aynı birimle çalışıyor; bir SCALE ve iki GEMVB op'u, ara scale'ler requant'lara katlanıyor. Birim yalnız modelde GEMVB op'u varsa sentezleniyor. İlk sürüm çok LUT tutuyordu, 14 709; akümülatörleri INT8 bankasından ayırıp 20 bite indirince 11 153'e düştü.

36. Kartta: düşük bitli modeller 1,7–3,6 kat hızlı ve BRAM'in yarısından azını kullanıyor; bedeli loss

Düşük bitli üç modeli de kart tasarımına koyduk ve diğer modeller gibi bit-exact doğruladık. Tablo sonuçları gösteriyor. Binary gpt2, INT8 gpt2'den 1,7 kat hızlı: token başına 8 713 yerine 4 983 cycle. LittleBit'li mlp 3,6 kat hızlı; mlp'nin ilk katmanı çok geniş olduğu için 64 hatlı birimden en çok o yararlanıyor. Binary stories260K 1,8 kat hızlı ve BRAM kullanımı 97,5 bloktan 56'ya iniyor. Kazancın neden 8 kat olmadığı ortada: yalnızca matris-vektör çarpımları hızlanıyor; norm, attention ve sampler aynı kalıyor. Bedelin iki tarafı var. Toplama hatları LUT'larla kurulduğu için LUT sayısı yüzde 35 ile 60 artıyor; çipte bunun için bolca yer var. Asıl bedel kalite: names'te binary 0,03, LittleBit 0,16 nat kaybettiriyor. stories260K'da ise binary 0,64 nat kaybettiriyor; 260 bin parametre bu görev için zaten sınırda ve weight'leri 1 bite indirmek çok ağır geliyor. Loss eğrisi son adımlarda hâlâ düşüyordu; daha uzun bir QAT bu farkı azaltabilir.

37. Doğrulama bir zincir: her halka bir öncekinin çıktısını referans alır, kart yerine Mac'te biter

Doğrulamayı sekiz halkalı bir zincir olarak kurduk; her halka bir öncekinin çıktısını referans alıyor. Gri ilk iki halka yazılımda: bizim float modelimizin Karpathy'nin koduyla aynı sonucu verdiğini, 10 üzeri eksi 4 toleransla, ve integer modelin kalitesinin korunduğunu, loss farkıyla ölçüyoruz. Turuncu halkalar donanımın tasarımını doğruluyor: kartın tamamının RTL simülasyonu, rastgele düğme ve anahtar testleri ve bilinmeyen değerleri yakalayan 4-state simülasyon. Mavi halkalar araçları doğruluyor: sentez aracının ürettiği kapı listesinin simülasyonu ve yerleşimden sonra zamanlama analizi. Sekizinci halka kartın kendisi olurdu; kararımız gereği bunu yapmıyoruz. Neden bit-exact şart? Çünkü donanım da referans da aynı tamsayı işlemlerini yapıyor; çıktılar yaklaşık değil tam olarak aynı olmak zorunda. Yaklaşık bir karşılaştırma bir hatayı saklayabilir; bit-exact karşılaştırmada tek bir farklı token hatayı ortaya koyuyor. Kart olmadan bu neden yeterli? Tasarımda tek clock var ve her şey senkron. Netlist mantıksal olarak doğruysa ve zamanlama analizi her yolun bir clock periyoduna sığdığını gösteriyorsa, çipteki devre simülasyondaki gibi davranır. Endüstride de bir çip üretime gitmeden önce bu iki kanıta dayanılır.

38. Tam kart simülasyonu: testbench bir kullanıcı gibi düğmelere basar ve yalnız pinlerden okunabileni karşılaştırır

Doğrulamanın merkezindeki test, kartın tamamının simülasyonu. Burada önemli bir tasarım kararı var: testbench tasarımın içine bakmıyor, gerçek bir kullanıcı gibi davranıyor. Reset'i bırakıyor, ilk koşunun bitmesini bekliyor, sonra düğmelere basıyor ve yalnızca kartın pinlerinden okunabilecek şeyleri karşılaştırıyor. Soldaki tablo karşılaştırılanları gösteriyor: üretilen her token, kart üstü kontrolün sonucu, ekrandaki metnin tamamı, VGA pinlerinden yakalanan bir kare, 7-segment gösterge ve LED'ler; ayrıca her cycle'da bütün pinlerin bir imzası tutuluyor, bu sonraki netlist karşılaştırmasında işe yarıyor. Her model için dört koşu yapılıyor: otomatik sampling koşusu, orta düğmeyle ikinci bir koşu, greedy modda yeniden tohumlanmış bir koşu ve ekran temizleme. Sağda stories260K için gerçek çıktı: 256 token aynı, hash aynı, ekran, durum satırı, LED'ler ve 7-segment doğru, VGA karesinde sıfır piksel fark. 18 modelin hepsi geçiyor; toplamda 11 241 token.

39. Rastgele ve köşe durumu testleri, normal kullanımda görünmeyen iki gerçek RTL hatası buldu

Normal kullanımla yapılan testler, kullanıcının makul davrandığını varsayar. Gerçek kullanıcılar ise düğmelere koşu ortasında basar, reset'e yanlış anda basar. Bunun için iki tür test yazdık. Rastgele testlerde anahtar ayarları ve düğme dizileri rastgele seçiliyor, basışlar koşunun ortasına da denk geliyor, CPU reset de dahil: toplam 655 koşu ve yaklaşık 121 bin token; 32 farklı sampling ayarının 30 ile 32'si kapsandı ve hepsi referansla aynı. Ayrıca 16 köşe durumu elle yazıldı; mesela koşu sırasında basılan düğme kuyruğa girmeli, reset sonrasında yarım kalan koşunun token'ları referansın başlangıcıyla aynı olmalı. Üçüncü test 4-state simülasyon: Verilator her biti 0 ya da 1 kabul ediyor; Icarus ise bilinmeyen değeri, X'i, temsil ediyor ve reset'i unutulmuş register'ları yakalıyor. Bu testler iki gerçek hata buldu. Birincisi, kuyruktaki bir koşunun başladığı cycle'a denk gelen bir düğme basışı kayboluyordu. İkincisi, LED'i yakıp söndüren sayacın reset'i yoktu; Verilator her biti 0 ya da 1 saydığı için görünmüyordu, 4-state simülasyonda ise LED X kaldı ve reset düğmesi bu sayacı sıfırlamıyordu. İkisi de düzeltildi ve bütün testler yeniden geçti.

40. Netlist simülasyonu: RTL doğru olsa bile araçlar hata yapabilir; üç araç hatası bulduk

RTL'in doğru olması, çipe giden devrenin doğru olduğu anlamına gelmiyor; arada RTL'i kapılara çeviren araçlar var ve onlar da hata yapabiliyor. Bu yüzden sentez aracının ürettiği kapı listesini, yani netlist'i de simüle edip RTL ile pin pin karşılaştırdık. Üç gerçek araç hatası bulduk. Birincisi sv2v'de: SystemVerilog'u Verilog'a çeviren bu araç, signed bir dizi elemanına uygulanan boyut cast'ini işaretsiz bir ifadeye çeviriyordu. Sonuç olarak attention çıkışındaki aritmetik kaydırma mantıksal kaydırmaya dönüşüyor ve negatif değerler 127'ye doyuyordu; açık bir signed ile çözdük. İkincisi Yosys'te: iki optimizasyon adımı tek başına doğru, ama art arda çalışınca attention skorundaki dinamik bayt seçimini bozuyordu; RTL'i, önce head'in dilimini seçip sonra sabit indeks kullanacak şekilde yeniden yazdık. Üçüncüsü yine Yosys'te: requant çarpımını DSP bloğuna eşlerken işaret uzatmasını düşürüyor. Bunu tek başına gösteren küçük bir test yazdık; çözüm olarak Yosys akışında çarpıcıları LUT'lara koyuyoruz. Sonuçta gerçek üst modülün netlist'i, MMCM ve BRAM'ler dahil, üç modelde her pinde ve her cycle'da RTL ile aynı. Verilator ve Vivado SystemVerilog'u doğrudan okuduğu için ilk iki hatadan etkilenmiyordu; yani bunlar yalnızca açık kaynak akışın hatalarıydı.

41. Vivado olmadan, Mac'te açık kaynak bir akışla: sentez, yerleşim, yönlendirme, zamanlama ve bitstream

Zamanlamayı ölçmek için devrenin çipe gerçekten yerleştirilmesi gerekiyor. Xilinx'in aracı Vivado macOS'ta çalışmıyor; bu yüzden tamamen açık kaynak bir akış kurduk. Yosys RTL'i LUT, flip-flop ve BRAM'lerden oluşan bir netlist'e çeviriyor. nextpnr bu netlist'i çipin fiziksel kaynaklarına yerleştiriyor, aralarındaki kabloları yönlendiriyor ve çipin gecikme modeline göre her yolun gecikmesini hesaplıyor; Fmax bu aşamada çıkıyor. Sonra açılacak bütün özellikler FASM adlı bir metin dosyasına yazılıyor ve Project X-Ray araçları bunu çipin yapılandırma belleğinin satırlarına, sonra da .bit dosyasına çeviriyor. Yolda dört sorun çıktı. Mac'in yeni SDK'sını Xcode'un linker'ı okuyamıyordu; Xcode'un kendi SDK'sını kullanarak çözdük. nextpnr'da pin atamasını boş gösteren bir hata bulduk: PACKAGE_PIN değerini boşalmış bir bellek referansından kopyalıyor; kısıt dosyasında doğrudan LOC yazarak aştık. nextpnr 50 megahertz tutmadığında hiçbir rapor yazmadan duruyordu; artık her durumda raporu yazdırıyoruz ve tutmazsa Vivado betiğimizdeki gibi 25 megahertz'te yeniden kuruyoruz. Dördüncüsü daha önce bahsettiğim Yosys DSP hatası: bitstream'de çarpıcılar LUT'larda, yani netlist simülasyonunda bit-exact olduğunu gösterdiğimiz netlist. bigram için bütün akış bir dakika sürüyor.

42. Fmax: makemore'un üç modeli 50 MHz'i tutuyor; transformer'lar 26–31 MHz'te kalıyor ve 25 MHz'te çalışıyor

Sonuçlar bu grafikte. Her çubuk, modelin 50 megahertz hedefiyle yapılan yerleşimde ulaştığı Fmax; iki dikey çizgi 25 ve 50 megahertz. Mavi çubuklar 50 megahertzi tutan modeller: bigram 71, mlp 66, rnn 67 megahertz. Bunlar en basit üç model; GEMV dışında neredeyse hiç op'ları yok. Turuncu çubuklar tutmayanlar. Transformer'lar 26 ile 31 megahertz arasında kümeleniyor; çünkü hepsi aynı attention ve norm yollarını kullanıyor ve kritik yol orada. Bunlar 25 megahertz için yeniden kuruldu ve hepsi orada zamanlamayı tutuyor. İki model, bow ve LittleBit'li mlp, 50 megahertzi kıl payı kaçırıyor: 47,7 ve 49,9. Bir bulgu daha var: aynı tasarımın Fmax'ı yerleşimin rastgele tohumuna göre oynuyor. bow'un 25 megahertz kurulumu bir tohumla 50,1, ötekiyle 44 megahertz verdi; 50

megahertz denemeleri ise 48,2 ve 47,7. mlp-lb'de 49,4 ve 49,9. Yani bu iki model bir pipeline değişikliği olmadan da başka bir tohumla 50 megahertzi tutabilir. İki TinyStories modeli, ts_gpt2 ve ts_microgpt, yerleşemedi: Yosys bunların belleğini çok sayıda küçük RAMB18 bloğuna bölüyor, BRAM yerlerinin yüzde 85'i doluyor ve nextpnr'ın yerleştiricisi iki farklı tohumla da 40 dakikada ilerleyemedi. Bunun çözümü bellek düzenini değiştirmek; sonraki adımlar arasında.

43. Kritik yol: oku, çarp, yaz tek cycle'da; çarpmalar LUT'ta olunca bu 37 ns sürüyor

Neden 50 megahertz tutmuyor? nextpnr'ın kritik yol raporu bunu açıkça gösteriyor. Engine bir op'u yürütürken her cycle'da şunu yapıyor: bir sayaçtan vektör tamponunun adresini hesaplıyor, tamponu okuyor, okunan değerlerle çarpma ve toplama yapıyor, sonucu doyurup tampona geri yazıyor; hepsi tek bir cycle'da. Tamponlar LUT'lardan yapılmış küçük RAM'ler ve okuması asenkron, yani okuma da aynı yolun parçası. Çarpmalar da Yosys'in DSP hatası yüzünden DSP bloklarında değil LUT'larda ve uzun carry zincirleri oluşturuyor. stories260K'da en uzun yol 36,7 nanosaniye ve 54 adım: bunun yalnızca 9,7 nanosaniyesi mantık, 26,9'u kablolama, yani sinyallerin çipin bir ucundan diğerine gitmesi. 50 megahertzin periyodu 20 nanosaniye; yol buna sığmıyor. 25 megahertzin 40 nanosaniyelik periyoduna ise 3,3 nanosaniye payla sığıyor. Yolun tam yeri modele göre değişiyor: transformer'larda attention'ın head dilimi, binary gpt2'de norm'un çıkış çarpımı; ama kalıp hep aynı: oku, çarp, yaz, tek cycle'da. Çaresi de belli: okuma ile hesabın arasına bir register koymak. Yol ikiye bölünür, op'lar bir cycle gecikir; hedefimiz 50 megahertz.

44. Yerleşimden sonra kaynaklar: en büyük model bile çipin üçte birinden azını kullanıyor

Yerleşimden sonra kaynak kullanımı bu. Sayılar nextpnr'dan. LUT sütunu, çipteki 126 800 LUT yuvasından kullanılanları gösteriyor. En büyük model stories260K'nın binary sürümü bile yüzde 31'de; INT8 stories260K yüzde 27'de. Yani mantık tarafında çipte bolca yer var. Bunun bir ayrıntısı önemli: DSP sütunu hep sıfır, çünkü Yosys'in DSP hatası yüzünden bütün çarpıcıları LUT'lara koyduk. Yosys'in DSP kullanan tahmini stories260K için 9 bin LUT ve 66 DSP diyordu; çarpıcılar LUT'a gidince LUT kullanımı birkaç kat artıyor ve kritik yol da uzuyor. BRAM tarafında sayılar Yosys tahminleriyle birebir aynı: stories260K yüzde 72, binary sürümü yüzde 41. Binary modellerin flip-flop sayısı belirgin şekilde yüksek; 64 hatlı 1 bitlik birimin akümülatörleri bunlar.

45. Gerçek hız = clock / (cycle / token): stories260K kartta saniyede 496 token yazıyor, 256 token'lık hikâye yarım saniye

Şimdi iki ölçümü birleştirebiliriz. Gerçek hız, clock frekansının token başına cycle sayısına bölümü. Token başına cycle tam kart simülasyonundan geliyor ve sampler ile ekran beklemesini de içeriyor; clock ise yerleşimde zamanlamanın tuttuğu frekans. En hızlısı bigram: saniyede 318 bin token, çünkü bir token için yalnızca bir tablo okuyor. makemore'un 50 megahertzde çalışan mlp ve rnn modelleri saniyede 7 500 token civarında. Transformer'lar 25 megahertzde saniyede 2 300 ile 2 900 token arasında. Karpathy'nin stories260K modeli saniyede 496 token yazıyor; 256 token'lık bir hikâye yarım saniyede bitiyor. Düşük bitli modeller belirgin şekilde hızlı: binary gpt2 saniyede 5 bin, LittleBit'li mlp 13 bin, binary stories260K 893 token. Ölçek vermek için: bir insan saniyede birkaç token okuyabiliyor; kart en yavaş modelde bile okuma hızımızın yüz katından hızlı yazıyor. Bu yüzden

karta, üretimi gözle izleyebilmek için saniyede yaklaşık 25 token'lık bir daktilo modu da koyduk. Pipeline ile 50 megahertze çıkabilirsek 25 megahertzdeki bütün sayılar iki katına çıkar.

46. Kalite ve hız birlikte: names'te en iyi loss gru'da, en hızlı INT8 model rnn; LittleBit hızı loss ile ödüyor

Son olarak bütün aileyi iki eksende karşılaştıralım. Dikey eksen kalite: kartın çalıştırdığı integer programın test loss'u; yukarı gittikçe daha iyi. Yatay eksen hız: kartta saniyede üretilen token, logaritmik ölçek; sağa gittikçe daha hızlı. Yani sağ üst köşe en iyi. Mavi noktalar makemore modelleri, turuncular transformer'lar, kareler düşük bitli modeller. Bu ölçekte, isim üretiminde, en iyi loss gru'da; ama gru 25 megahertzde çalıştığı için saniyede 3 800 token. rnn ve mlp hem iyi loss'a sahip hem de 50 megahertzi tuttukları için saniyede 7 500 token civarında; bu ölçekte en iyi denge onlar. Transformer'lar sol üstte toplanıyor: loss'ları iyi ama norm ve attention yüzünden hem daha çok cycle harcıyorlar hem de 25 megahertzde kalıyorlar. Binary gpt2 transformer'ları hız ekseninde neredeyse iki kat sağa taşıyor, loss'ta küçük bir bedelle. LittleBit'li mlp en hızlısı, saniyede 13 bin token, ama loss'ta en büyük bedeli ödüyor. bigram ölçeğin çok dışında: saniyede 318 bin token, ama loss 2,45 ile en kötüsü. Bu grafik isim gibi kısa diziler için; TinyStories gibi uzun context'li görevlerde attention'ın payı büyüdüğü için transformer'ların durumu değişiyor.

47. Sınırlar: neyi kanıtladık, neyi henüz kanıtlamadık

Sonuçları değerlendirmeden önce sınırları açıkça söylemek istiyorum. Birincisi, fiziksel kart testi yapmadık; bu bir karardı. Kanıtımız simülasyon ve zamanlama analizi. Burada tek boşluk şu: yerleşimden sonraki son devreyi simüle edemiyoruz, çünkü nextpnr onu simüle edilebilir Verilog olarak yazamıyor; o aşama için zamanlama analizine güveniyoruz. İkincisi, ölçtüğümüz Fmax yaklaşık. nextpnr'ın kullandığı gecikme modeli Project X-Ray topluluğunun çipten çıkardığı bir model; Vivado'nun sonucu farklı olabilir. Ayrıca çarpıcılar LUT'larda olduğu için bu devre Vivado'nun DSP'li devresinden daha yavaş ve daha büyük. Üçüncüsü kapasite: çipin 607 kilobaytlık BRAM'i yaklaşık 300 bin parametreye kadar yetiyor. nanoGPT'nin ya da llm.c'nin hazır eğitilmiş modelleri on milyonlarca parametre; onlar için kartın DDR2 belleğinden weight okuyan farklı bir mimari gerekir. Dördüncüsü eksik parçalar: iki TinyStories modeli, ts_gpt2 ve ts_microgpt, bellekleri çok sayıda küçük BRAM bloğuna bölündüğü için yerleşemedi; ilk sunumun konusu olan minGPT'nin sıralama demosu bu kart tasarımında yok, eski hızlandırıcıdaydı; Larq'ın gerçek paketiyle sayısal karşılaştırma yapmadık, çünkü TensorFlow kurmadık; LittleBit'in iki yollu sürümü de donanımda yok, yalnızca tek yollu.

48. Sonraki adımlar: önce hız, sonra doğrulamanın son boşluğu, sonra kapsam

Sonraki adımları öncelik sırasıyla sıraladım. Birincisi hız: engine'de tampon okuması ile hesabın arasına bir register koymak. Bu, kritik yolu ikiye bölüyor ve hedefimiz 50 megahertz; bu da token/s'de iki kat demek. Bu bir RTL değişikliği olduğu için 18 modelin bütün simülasyonları yeniden geçmeli. İkincisi doğrulamanın son boşluğu: yerleşime giren netlist'i, gerçek clock ayarı ve gerçek debounce süreleriyle, 18 modelin hepsinde simüle etmek. Üçüncüsü DSP blokları: Yosys'teki hatayı geliştiricilere raporlamak ve hatanın görüldüğü requant çarpımı dışındaki çarpıcıları yeniden DSP'ye koymak. Bu hem LUT sayısını düşürür hem Fmax'ı artırır. Aynı adımda yerleşemeyen iki TinyStories

modelinin tablolarını, Yosys onları küçük bloklara bölmeyecek şekilde yeniden düzenlemek gerekiyor. Dördüncüsü kapsam: ilk sunumun konusu olan minGPT'nin sıralama demosunu 19. model olarak kart tasarımına eklemek; ekranda girdi dizisini ve sıralanmış çıktıyı göstermek. Beşincisi düşük bit tarafı: stories260K için daha uzun bir binary QAT, çünkü loss son adımlarda hâlâ düşüyordu; ve LittleBit'in iki yollu sürümünü donanımda denemek.

49. son

Toparlamak için dört cümle. Birincisi: Karpathy'nin dokuz dil modeli mimarisi ve hazır eğitilmiş stories260K modeli tek bir mikrokodlu integer engine'de çalışıyor; bir modelden ötekine geçerken devre değişmiyor, yalnızca BRAM'deki mikrokod ve weight'ler değişiyor. İkincisi: kart hiçbir bilgisayara bağlı olmadan çalışıyor; üretim döngüsü, integer sampling, token'ların metne çevrilmesi ve VGA ekranı FPGA'nin içinde. Kartın tamamının simülasyonunda 18 modelin hepsi Python'daki referansla bit bit aynı sonuç veriyor. Üçüncüsü: hocamın önerdiği yöntemlerden binary weight'ler 1,7 ile 3,6 kat hız ve yarı bellek getiriyor; LittleBit ise küçük modellerde scale maliyeti yüzünden ancak geniş katmanlarda binary'yi geçiyor. Dördüncüsü: Vivado olmadan, açık kaynak araçlarla Mac'te bitstream ürettik ve zamanlamayı ölçtük. 16 model yerleşti; makemore'un en basit üç modeli 50 megahertzde, diğerleri 25 megahertzde zamanlamayı tutuyor. Engine'e pipeline ekleyerek hedefimiz hepsini 50 megahertze çıkarmak. Teşekkür ederim; sorularınızı bekliyorum.